

Junior AI Engineer — Take-Home Assignments

18 Sept 2026

Purpose

We're hiring a junior AI engineer who will work mostly on our autonomous agent products. These three take-home assignments test the core skills that matter for that work: planning without a fixed script, using tools reliably, handling failure without crashing, and reasoning about state.

There are 3 total at least two assignments need to be submitted

General rules:

- Submit as a GitHub repo (public or shared with us), not a zip
- Each assignment in its own folder with its own README covering setup and how to run it
- Use of AI coding assistants (Copilot, Claude, Cursor, ChatGPT) to write code is allowed and expected — this is normal workflow, not cheating
- If a requirement is unclear, state your assumption in the README and move on — don't block on it

Framework Requirement

All three assignments must be built using **both LangGraph or LangChain — at least any one of the frameworks**, without fail. This applies to all three, including Assignment 3 (use LangGraph's built-in state/checkpointing rather than hand-rolling your own).

You are welcome to use an LLM (ChatGPT, Claude, etc.) to help you design or write the solution. That's allowed and expected — what matters is that you understand the code you submit and can explain any decision in it during the follow-up conversation.

Free LLM Access

Pick any one of these — model choice isn't graded, agent design is:

Option	Notes
Google AI Studio (Gemini)	Free tier, generous daily quota, no card required. Recommended default.
Groq	Free tier, very fast inference, open models (Llama, etc).
Ollama	Runs fully local, zero cost, zero signup. Needs a reasonably capable laptop.

Already have Anthropic or OpenAI credits? Those work fine too.

Assignment 1 : Tool-Using Research Agent

Context: We build autonomous agents that operate with minimal supervision. This tests whether your agent can decide its own path to an answer, rather than following a script you wrote for it.

Task: Pick one open-ended question that can't be answered from a single lookup — it needs reasoning across multiple pieces of information. Examples: "What's the best caching strategy for a read-heavy API with 10k req/sec?" or "Compare two approaches to X and recommend one for Y use case."

Build a single agent (in LangGraph / LangChain) that:

- Plans its own steps toward an answer — no hardcoded "step 1, step 2" script
- Calls **at least 2 distinct tools** — no upper limit, use more if the task benefits from it (web search, calculator, file read, code execution, custom API, etc.)
- Decides on its own when it has enough to answer — not a fixed number of iterations

Constraints:

- Max 6 tool calls per run — the agent must track its own count and stop itself if it hits the limit without an answer
- Every step must log what it decided to do, why, and the result — a reasoning trace, not just tool inputs/outputs
- One tool call must be made to fail (mock a bad response: timeout, empty result, malformed data). The agent must notice and adapt — not crash or silently continue as if nothing happened

Deliverables:

- Source code
- Short README: how to run it, which tools it uses, what question it answers
- Transcript 1: clean run, no failures, final answer
- Transcript 2: same or similar run, with the mocked tool failure visible and how the agent handled it

What we're grading: Does the agent actually plan, or is it a disguised fixed pipeline? Is the reasoning log readable? Does it handle failure gracefully?

Assignment 2 : Multi-Agent Task with Review

Context: Real products often need one agent to check another's work. Here we're testing the handoff between two agents and whether the system reports a clear verdict — not building a negotiation loop.

Task: Pick a small task with a clear "good enough" bar — e.g. "write a Python function for X and have it reviewed", "draft a 100-word product description and have it checked for accuracy", "plan a 3-day itinerary and have it checked for feasibility."

Build two agents (in LangGraph / LangChain) in a single chain:

- **Agent A (worker):** produces one attempt
- **Agent B (reviewer):** reviews that single attempt against clear criteria (you define the criteria) and returns a verdict: approved, or rejected with specific reasons

No revision loop — B reviews once, chain ends. Result is either "approved" + final output, or "rejected" + Agent B's specific reasons.

Constraints:

- Agent B's criteria must be concrete enough that its verdict is explainable, not a vague "looks fine"
- Report total LLM calls and (if easy to get) total tokens used at the end of the run

Deliverables:

- Source code
- Short README: the task, what Agent B's approval criteria are
- Transcript 1: a run where B approves
- Transcript 2: a run where B rejects, showing B's specific reasons for rejection (pick a weaker Agent A output or stricter criteria to force this)

What we're grading: Does Agent B give useful, specific feedback, or just approve/reject with no reasoning? Is the handoff clean? Is the final report clear either way?

Assignment 3 : Resumable Agent with Basic Self-Check

Context: Long-running agent tasks can get interrupted. This tests whether you can save progress and pick back up correctly.

Task: Pick a small task with 4–5 items to process one at a time — e.g. "summarize 4 files", "process 4 rows of data and flag anything unusual", "answer 4 sub-questions on a topic."

Build an agent (in LangGraph / LangChain) that:

- Processes items one at a time, in order
- Persists progress after each item is done
- Can be stopped partway (Ctrl+C, or a temporary line like `if item == "item3": exit()`) and, when re-run, picks up from where it left off — skips items already completed
- After all items are done, runs one simple check: re-reads the final results and confirms each one is non-empty and roughly matches what it should be (e.g. ask the LLM once: "does this summary match this file, yes or no"). Deliberately make one result wrong/blank on purpose, so we can see the check actually catches it

Constraints:

- Keep the persisted state simple and readable — this isn't about clever state design, just correct save/resume
- Report total LLM calls made at the end

Deliverables:

- Source code
- Short README: the task, what the saved state looks like, how to trigger the stop/resume
- Transcript/log: run → stopped partway → re-run → completes, showing which items were skipped vs newly processed
- Transcript/log: the check step catching the one bad result

What we're grading: Does resume actually skip completed work? Does the check step actually flag the bad result? Is the code simple and readable?

Submission

- Submit as a GitHub repo (public or shared with us)
- Folders: assignment-1/, assignment-2/, assignment-3/, each with its own README
- Code must run from a clean clone following your own README
- Transcripts/demos requested in each assignment must be included in the repo, not just described